

Module 4: Observability Dashboard

Dashboard UX Principles

Know your user:

- ▶ Instructors want: Quick overview, actionable items
- ▶ Not interested in: Technical metrics, raw data

Information hierarchy:

- ▶ Most important info at top-left
- ▶ Progressive disclosure (summary -> details)

Key metrics first:

- ▶ Today's sessions
- ▶ Check-in rate
- ▶ Flagged items requiring action

Reduce cognitive load:

- ▶ Use consistent layouts
- ▶ Group related information

Data Visualization Best Practices

Choose the right chart type

- ▶ Line chart: Trends over time
- ▶ Bar chart: Comparisons between categories
- ▶ Pie chart: Parts of a whole (use sparingly)
- ▶ Table: Precise values, sortable data

Design principles:

- ▶ Start Y-axis at zero (no misleading scales)
- ▶ Use color meaningfully (red=bad, green=good)
- ▶ Label clearly - no guessing required
- ▶ Avoid chartjunk (unnecessary decoration)

Mobile considerations: Tables should scroll horizontally

Real-Time Updates Architecture

Options for real-time data:

- ▶ Polling: Client requests every N seconds
- ▶ WebSockets: Persistent two-way connection
- ▶ Server-Sent Events (SSE): Server pushes updates

Polling (simplest):

- ▶ `setInterval(() => fetchData(), 30000)`
- ▶ Good enough for dashboards
- ▶ Less server complexity

When to use WebSockets:

- ▶ Very frequent updates (< 1 second)
- ▶ Two-way communication needed
- ▶ Many simultaneous connections

Metrics, Monitoring & Alerting

Types of metrics

- ▶ Business: Check-in rate, attendance %
- ▶ Technical: Response time, error rate

Prometheus + Grafana stack:

- ▶ Prometheus: Collects and stores metrics
- ▶ Grafana: Visualizes metrics, creates dashboards

Key metrics for attendance:

- ▶ Check-ins per session
- ▶ Average risk score distribution
- ▶ Flagged vs. approved ratio

Alerts:

- ▶ High flagged rate -> possible issue
- ▶ Many failed logins -> possible attack

Role-Based Access in Dashboards

Different users need different views

- ▶ Student: Own check-in history only
- ▶ TA: Session check-ins for courses they assist
- ▶ Instructor: Full course analytics, student details
- ▶ Admin: System-wide metrics, all courses

Implementation:

- ▶ Check JWT role on every request
- ▶ Filter data based on permissions
- ▶ Hide UI elements user can't access

Don't rely on UI hiding alone:

- ▶ Backend must enforce permissions
- ▶ Hidden button != secure endpoint

Export & Reporting Features

Export formats:

- ▶ CSV: Universal, opens in Excel
- ▶ JSON: For programmatic use
- ▶ PDF: For official records

CSV best practices:

- ▶ Include headers
- ▶ Handle special characters (commas, quotes)
- ▶ Use UTF-8 encoding

Attendance report columns

- ▶ Student name, email, course
- ▶ Session date, check-in time
- ▶ Status (approved/flagged/rejected)
- ▶ Risk score (for instructor reference)